Solving Stiff ODEs

Wenchong Huang

Department of Mathematics, Zhejiang University.

June 6th, 2023.

1. An overview
2. An implemention of implicit RK methods
3. An introduction to stiff equations
4. An adaptive ESDIRK method
5. The Radau-IIA methods

1. An overview
2. An implemention of implicit RK methods
3. An introduction to stiff equations
4. An adaptive ESDIRK method
5. The Radau-IIA methods

An overview of all methods

Method	Abbreviation	Approch	Step Size	Stages	Order	A-Stable	B-Stable	L-Stable
Classical RK	CLRK	Explicit	Constant	4	4			
ESDIRK	ESDIRK	Implicit	Constant	6	4	Yes	Yes	Yes
Gauss-Legendre	GLRKs	Implicit	Constant	s	$2s$	Yes	Yes	
Radau-IIA	RIIARKs	Implicit	Constant	s	$2s-1$	Yes	Yes	Yes
Fehlberg	FB4(5)	Explicit	Adaptive	6	4			
Dormand-Prince	DOPRI5	Explicit	Adaptive	7	5			
Dormand-Prince 8(7)	DOPRI8	Explicit	Adaptive	13	8			
Adaptive ESDIRK	A-ESDIRK	Implicit	Adaptive	6	4	Yes	Yes	Yes
Adaptive Gauss-Legendre	A-GLRKs	Implicit	Adaptive	s	$2s$	Yes	Yes	
Adaptive Radau-IIA	A-RIIARKs	Implicit	Adaptive	s	$2s-1$	Yes	Yes	Yes

1. An overview
2. An implemention of implicit RK methods
3. An introduction to stiff equations
4. An adaptive ESDIRK method
5. The Radau-IIA methods

A simple iterative method

As known to us, we cannot compute $\mathbf{y}_i$ directly in implicit RK methods. For the ESDIRK method, we have

$$\mathbf{y}_i = \mathbf{f}(\mathbf{z}_i + \gamma \mathbf{y}_i, t_n + c_i k),$$

where $\mathbf{z}_i$ is a konwn vector. We can set $\mathbf{y}_i \leftarrow \mathbf{f}(\mathbf{U}^n, t_n)$ initially. Then update $\mathbf{y}_i$ for several times by the above equation. And we can compute $\mathbf{y}_1, ..., \mathbf{y}_4$ in order.

For collocation methods, we have

$$\mathbf{y}_i = \mathbf{f}\left(\mathbf{U}^n + k\sum_{j=1}^{s} a_{ij}\mathbf{y}_j, t_n + c_i k\right), \quad i = 1, ..., s.$$

We can set $\mathbf{y}_i \leftarrow \mathbf{f}(\mathbf{U}^n, t_n)$ $(i = 1, ..., s)$ initially. Then update $\mathbf{y}_1, ..., \mathbf{y}_s$ in order for several times.

Remark. In the case that $\mathbf{f}(\mathbf{U}, t) = A\mathbf{U} + \mathbf{b}(t)$, the $\mathbf{y}_i$s can be computed directly by solving the linear systems. Suppose $\mathbf{U}^n$ is a vector of dimension m. The ESDIRK method needs to solve an $m \times m$ system for s times, while the collocation methods need to solve an $sm \times sm$ system for one time. The ESDIRK method will be much faster.

The Newton method

Sometimes the simple iterative method mentioned above does not work. So we need another method. Rewrite

$$\mathbf{y}_i = \mathbf{f}\left(\mathbf{U}^n + k\sum_{j=1}^{s} a_{ij}\mathbf{y}_j, t_n + c_i k\right), \quad i = 1, ..., s$$

as

$$\mathbf{F}(\mathbf{p}) = \mathbf{0}, \quad \text{where} \quad \mathbf{p} = (\mathbf{y}_1^T, ..., \mathbf{y}_s^T)^T.$$

The Newton method gives that

$$\mathbf{p} \leftarrow \mathbf{p} - J(\mathbf{F}(\mathbf{p}))^{-1}\mathbf{F}(\mathbf{p}),$$

where J is the Jacobi matrix. To get the jacobi matrix, the derivatives should be computed. If someone uses

$$g'(x) \approx \frac{g(x+h) - g(x-h)}{2h}$$

to compute the numerical derivative, then h should be carefully chosen. The best choice is

$$h = \sqrt[3]{\frac{3\epsilon}{M}}.$$

where ϵ is the machine precision and $M = \sup|g'''(x)|$.

1 An overview

2 An implemention of implicit RK methods

3 An introduction to stiff equations

4 An adaptive ESDIRK method

5 The Radau-IIA methods

Definition

A **stiff equation** is a differential equation for which certain numerical methods for solving the equation are numerically unstable, unless the step size is taken to be extremely small.[1]

Stiff equations are equations where certain implicit methods, in particular BDF, perform better, usually tremendously better, than explicit ones.[2]

Stiff equations are problems for which explicit methods don't work.[3]

An IVP is said to be **stiff in an interval** if for some initial condition any numerical method with a bounded RAS is forced to use a time-step size that is unneccesarily and excessively smaller than the time scale of the true solution of the IVP.[4]

[1] From Wikipedia.
[2] Form Curtiss and Hirschfelder.
[3] From Hairer.
[4] From the textbook.

A sample stiff problem

The IVP in Example 11.144

$$u'(t) = \lambda(u - \cos t) - \sin t, \quad u(0) = \eta, \tag{1}$$

where $\lambda = -10^6$ and $\eta = 1.5$ is a stiff problem.

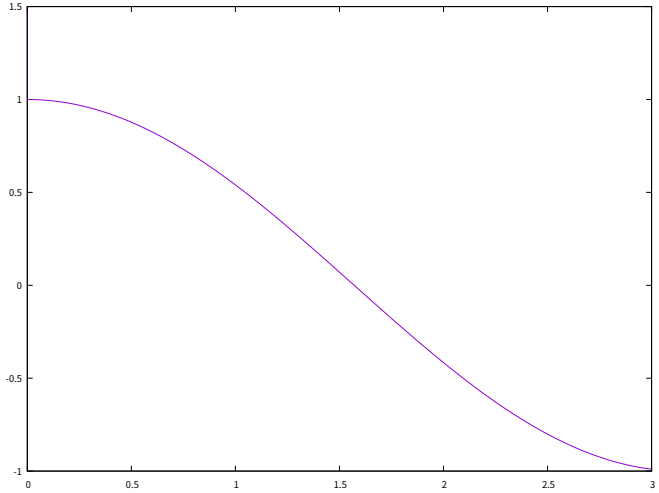


图 1: The exact solution to (1).

A sample stiff problem

The classical RK method goes to ∞ immediately.

```
0 1.5
0.1 2.08324989835e+18
0.2 8.67986071191e+36
0.3 3.61646396996e+55
0.4 1.50679971489e+74
0.5 6.27808101963e+92
0.6 2.61576245997e+111
0.7 1.08985743025e+130
0.8 4.5408909886e+148
0.9 1.89196223267e+167
1 7.88286065188e+185
```

图 2: The numerical results given by the classical RK method.

The Gauss-Legendre RK methods give the following results.

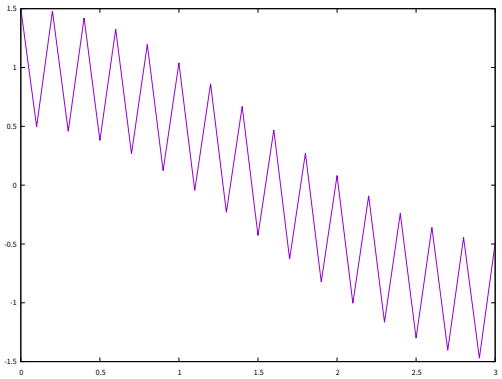


图 3: $s = 1$

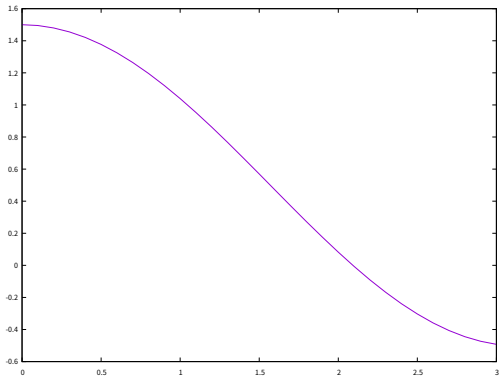


图 4: $s = 2$

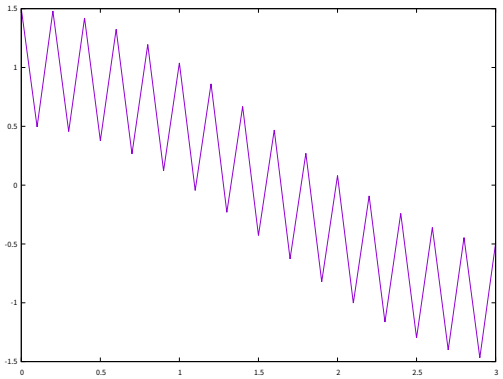


图 5: $s = 3$

A sample stiff problem

The Dormand-Prince method also performs badly. It used 903579 time steps to solve (1) with the tolerance 10^{-6}.

To solve (1), we need an L-stable method. The backward Euler method or the ESDIRK method in the textbook could be used, which have order 1 and 4 respectively.

A famous stiff problem - Van der Pol equation

The Van der Pol equation is

$$u'' = \lambda(1-u^2)u' - u, \quad u(0) = 2, \quad u'(0) = 0, \tag{2}$$

where $\lambda = 1000$.

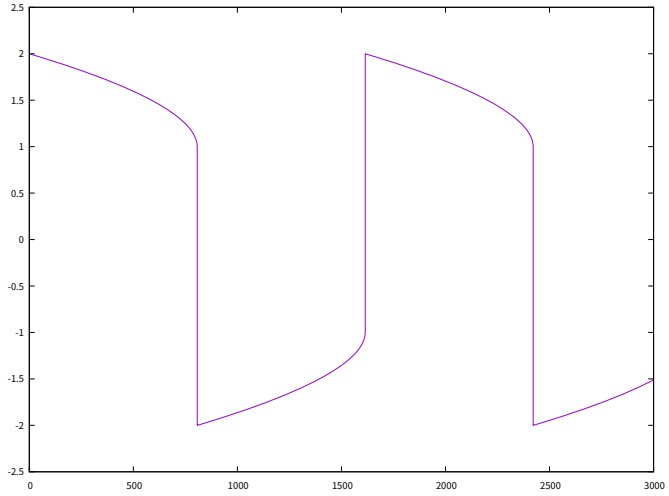


图 6: The exact solution to (2).

No method in the assignment can solve the problem well!

1 An overview

2 An implemention of implicit RK methods

3 An introduction to stiff equations

4 An adaptive ESDIRK method

5 The Radau-IIA methods

Introduction

Our motivation is to solve the Van der Pol's equation. Use Richardson extrapolation

$$\begin{aligned}
\mathbf{U}^{n+\frac{1}{2}} &= \mathbf{U}^n + \frac{k}{2}\mathbf{\Phi}\left(\mathbf{U}^n, t_n; \frac{k}{2}\right), \\
\tilde{\mathbf{U}}^{n+1} &= \mathbf{U}^{n+\frac{1}{2}} + \frac{k}{2}\mathbf{\Phi}\left(\mathbf{U}^{n+\frac{1}{2}}, t_n + \frac{k}{2}; \frac{k}{2}\right), \\
\hat{\mathbf{U}}^{n+1} &= \tilde{\mathbf{U}}^{n+1} + \frac{1}{2^p - 1}\left(\tilde{\mathbf{U}}^{n+1} - \mathbf{U}^{n+1}\right)
\end{aligned}$$

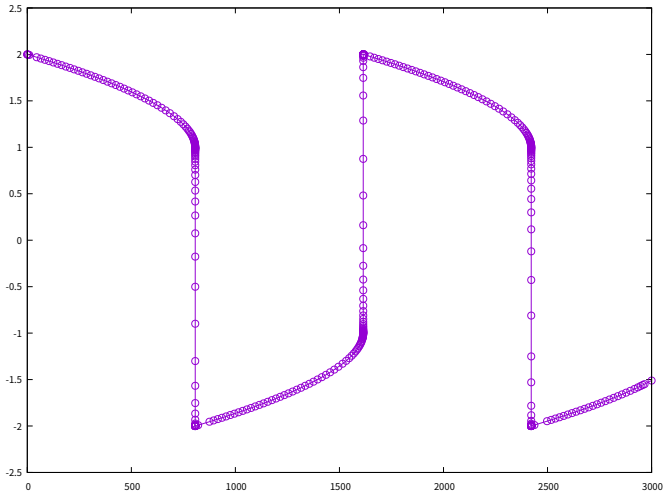
to get $\hat{\mathbf{U}}^{n+1}$. The remaining steps are the same as the Formula 11.234.

Numericacl experiment

Use the adaptive ESDIRK method to solve the Van der Pol equation. The following table shows the steps and CPU times it takes.

Tolerance	10^{-4}	10^{-6}	10^{-8}	10^{-10}
Steps	376	638	1273	3382
CPU Time(s)	0.375	0.296	0.161	0.254

The following figure is the solution with the tolerance 10^{-4}.



1 An overview

2 An implemention of implicit RK methods

3 An introduction to stiff equations

4 An adaptive ESDIRK method

5 The Radau-IIA methods

A review of Gauss-Legendre methods

Recall that how we construct the s-stages Gauss-Legendre method.

A review of Gauss-Legendre methods

Recall that how we construct the s-stages Gauss-Legendre method.

Let $c_1, ..., c_s$ be the roots of $P_s(x)$, where P_s is a polynomial of degree s and has distinct roots (see the textbook). Then we let

$$a_{ij} = \int_0^{c_i} l_j(x)\mathrm{d}x$$
$$b_j = \int_0^1 l_j(x)\mathrm{d}x$$

where l_j is the elementary Lagrange interpolation polynomial.

It constructs a s-stages, $(2s)$th-order collocation method, which is A-stable and B-stable but not L-stable. The order could be arbitrary high without extra artificial work.

We can derive adaptive Gauss-Legendre methods immediately with Richardson extrapolation.

Solving the stiff problem with adaptive Gauss-Legendre methods

Adaptive Gauss-Legendre methods perform well in the Van der Pol equation. The solution with the tolerance 10^{-4} are shown as follows.

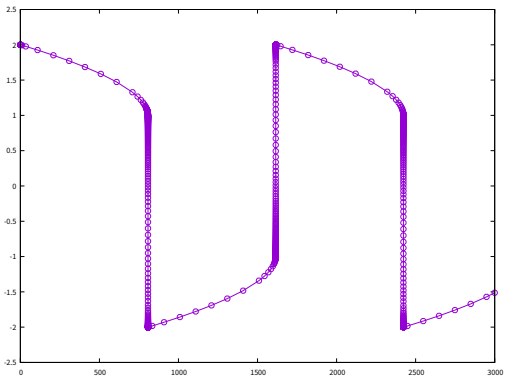


图 7: $s = 1,\ n = 918$

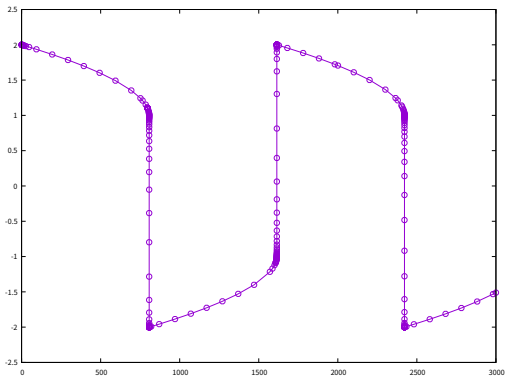


图 8: $s = 2,\ n = 269$

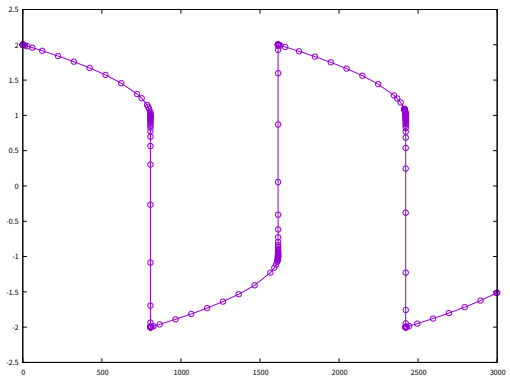


图 9: $s = 3,\ n = 207$

Solving the stiff problem with adaptive Gauss-Legendre methods

Adaptive Gauss-Legendre methods perform well in the Van der Pol equation. The solution with the tolerance 10^{-4} are shown as follows.

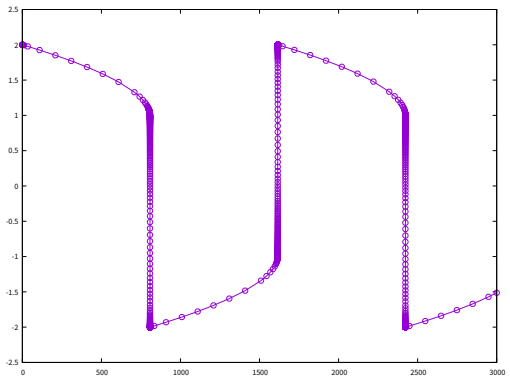


图 7: $s = 1,\ n = 918$

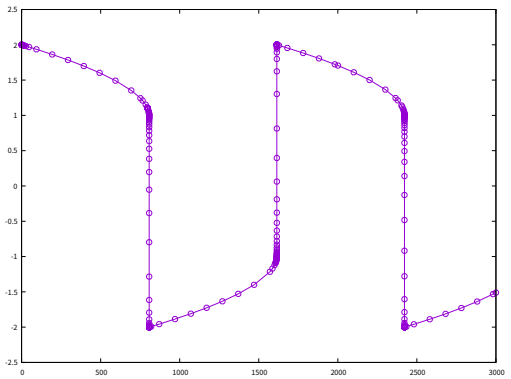


图 8: $s = 2,\ n = 269$

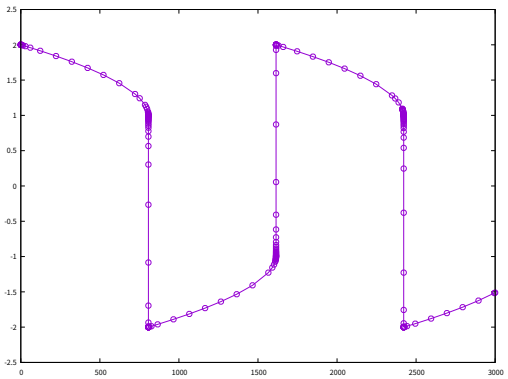


图 9: $s = 3,\ n = 207$

However, the adaptive 3-stages Gauss-Legendre method takes 5.46s to solve the problem (1) with the tolerance 10^{-13}. The adaptive ESDIRK method also takes 3.7s. We think they are too slow.

Constructing Radau-IIA methods

The motivation is to construct a high-order, adaptive and L-stable method.

Constructing Radau-IIA methods

The motivation is to construct a high-order, adaptive and L-stable method. Suppose the polynomial

$$Q_s(x) = \frac{\mathrm{d}^{s-1}}{\mathrm{d}x^{s-1}} x^{s-1}(x-1)^s.$$

It can be proved that Q_s has s distinct roots located in $(0, 1]$. Let $c_1, ..., c_s$ to be the roots of $Q(s)$ and then derive a collocation method.

The s-stages Radau-IIA method has $(2s-1)$th-order and is L-stable. And also we can derive adaptive Radau-IIA methods.

Solving stiff problems with adaptive Radau-IIA methods

Adaptive Radau-IIA methods perform well in the Van der Pol equation. The solution with the tolerance 10^{-4} are shown as follows.

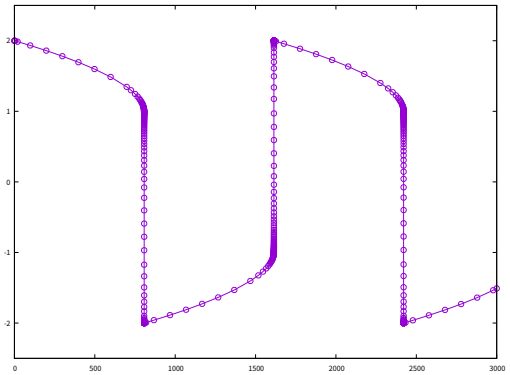


图 10: $s = 2,\ n = 421$

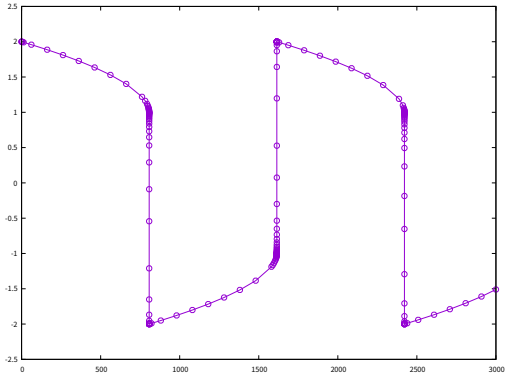


图 11: $s = 3,\ n = 200$

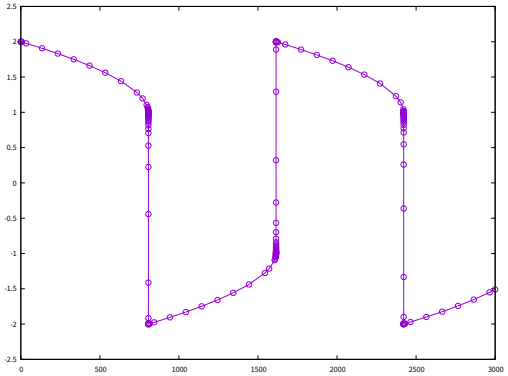


图 12: $s = 4,\ n = 170$

Solving stiff problems with adaptive Radau-IIA methods

Use the A-RIIARK3 method to solve the problem (1), comparing with the A-GLRK3 method as the following figures.

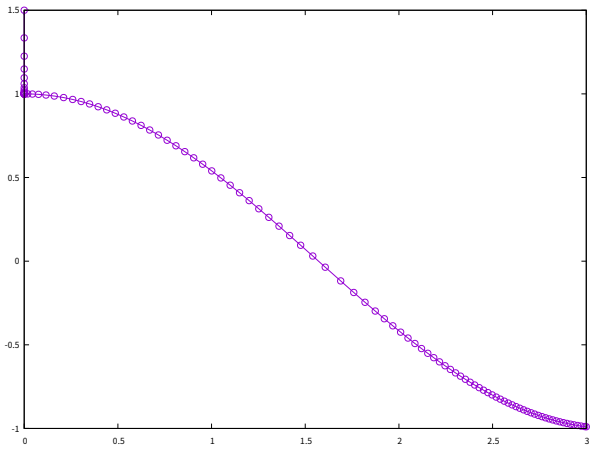


图 13: A-GLRK3, $n = 109$, 5.46s

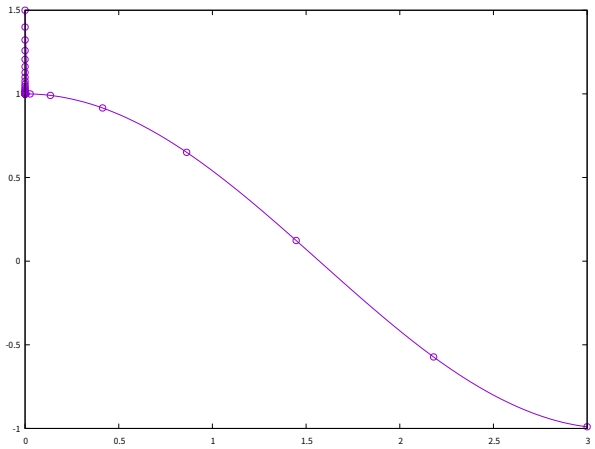


图 14: A-RIIARK3, $n = 43$, 0.769s

Thanks

Thaks for Your Attention!